人工智能第二次作业
MNIST 手写数字识别实验报告

钟锦程　2026210748　电子工程系

摘要

我用 PyTorch 在 MNIST 上实现了线性分类器、几种全连接神经网络（MLP）和卷积神经网络（CNN）。线性模型用作 baseline；MLP 比较隐藏层宽度与学习率；CNN 另外比较了训练时的轻量数据增强。训练集中的 6000 张图片专门作为验证集。最好的 MLP 在测试集上取得 97.70% 的准确率，普通 CNN 为 98.78%，增强 CNN 为 98.96%。我还用固定旋转平移和自己生成的字体数字检查了模型遇到分布变化时的表现。

目录

1　任务、环境与实验安排 **2**

2　数据处理、训练与指标 **2**

3　模型结构与作用 **3**

4　结果与参数对比 **4**

5　额外 baseline、数据增强和鲁棒性 **5**

6　问题与思考 **6**

7　心得与不足 **7**

1 任务、环境与实验安排

MNIST 中的每张图片是 28×28 的单通道灰度图，标签是 0 到 9 中的一个数字。模型读入图片后输出 10 个类别的分数，预测类别取分数最大的下标。完整数据包括 60000 张训练图片和 10000 张测试图片。

实验使用 macOS 26.6.2、Apple M5 Pro、24 GiB 内存、Python 3.11.15、PyTorch 2.14.1、torchvision 0.29.1。训练在 Apple MPS 上进行，绘图使用 Matplotlib 3.11.2。完整代码和复现说明放在单独的代码压缩包中。

原始训练集按随机种子 2026 固定划分：54000 张用于更新参数，6000 张用于挑选模型；官方测试集的 10000 张在训练结束后用于评估。线性模型、MLP 和 CNN 使用同一组下标与同样的标准测试集。这样在比较模型时，数据划分带来的差别会小一些。

2 数据处理、训练与指标

2.1 从附件到模型输入

程序优先从课程附件 `mnist.zip` 解压四个 IDX 文件；如果本地没有附件，会通过 torchvision 下载 MNIST。像素原本是 0 到 255 的整数，先除以 255 转到 0 到 1。划分训练集后，程序从本地训练图片的全部像素实际计算均值和总体标准差，再用这两个值归一化训练、验证和测试图片。设这些像素值为 x_i，总数为 $N = 54000 \times 28 \times 28$，计算方式是：

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i, \qquad \sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i-\mu)^2}, \qquad z = \frac{x-\mu}{\sigma}. \tag{1}$$

这一步让输入的大部分数值靠近 0，便于神经网络训练。实际运行得到 $\mu = 0.13059838$、$\sigma = 0.30804330$，用于归一化。增强 CNN 在读取训练图片时额外进行随机仿射变换。对宽 W、高 H 的图片，旋转角度和两个方向的位移分别从以下范围随机抽取：

$$\theta \sim U(-12^\circ, 12^\circ), \qquad u_x \sim U(-0.1W, 0.1W), \qquad u_y \sim U(-0.1H, 0.1H). \tag{2}$$

程序将位移 u_x, u_y 四舍五入为整数像素 Δ_x, Δ_y。设 p 是像素坐标，c 是图像中心，旋转与平移合起来可写成：

$$p' = R(\theta)(p-c) + c + \begin{pmatrix}\Delta_x \\ \Delta_y\end{pmatrix}, \qquad R(\theta) = \begin{pmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{pmatrix}. \tag{3}$$

MNIST 图片大小为 28×28，因此水平、竖直位移的采样范围均约为 ± 2.8 像素。每次读取训练图片都会重新抽样；验证集和测试集仍只进行前面的归一化。

2.2 训练、验证和停止规则

每次实验从固定随机种子开始初始化模型，用批大小 256 和 Adam 优化器，以交叉熵损失训练。默认最多训练 6 个 epoch。每轮结束后在验证集上计算准确率：若超过该实验此前的最好结果，就保存这一轮的权重；若连续两轮没有提高，就提前停止。最终测试使用保存的最好权重。MLP 的不同参数组合也只用验证准确率挑选。

2.3 准确率与宏平均 F1

准确率是预测正确的图片数除以总图片数。F1 需要先针对每个数字统计查准率与查全率：

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{4}$$

为了给十个数字同等权重，报告中的 F1 是 $\frac{1}{10}\sum_{c=0}^{9} F1_c$，即宏平均 F1。

3 模型结构与作用

下面给出不同模型的结构示意图。图中的数字表示张量尺寸。

3.1 线性分类器与 MLP

线性分类器把 784 个像素直接映射到 10 个类别，参数数目为 $784 \times 10 + 10 = 7850$。它可以衡量加入隐藏层究竟带来了多少收益。

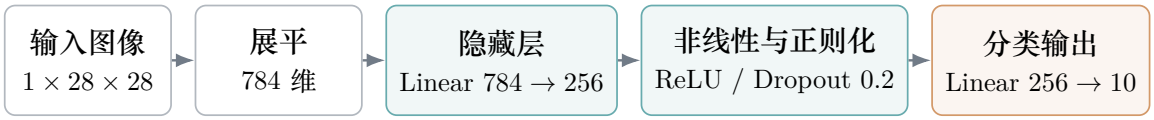


图 1: 按实际代码绘制的 MLP 结构图。线性 baseline 将展平后的 784 维像素直接映射为 10 类分数。

MLP 的隐藏层先组合像素信息，ReLU 提供非线性能力，Dropout 在训练时随机舍弃 20% 的隐藏单元，最后一层输出 10 个类别分数。上图使用宽度 256，对应 203530 个参数；比较实验还将隐藏层宽度改为 128。

3.2 CNN

CNN 保留图片的二维邻近关系。第一个 3×3 卷积产生 16 张特征图；池化将宽高减半。第二个卷积产生 32 张特征图，再池化到 7×7，最后由全连接层完成分类。两个 CNN 的结构和参数量完全相同，区别只在训练时是否增强图片。

作业 PDF 中推荐的 CNN 结构是用于处理句子的，似乎不能直接用来处理图片。只借鉴其中“卷积提取局部特征、池化汇总特征、全连接分类”的思路，用于二维数字图片的处理。具体采用串联的二维卷积与 2×2 空间池化。网络末端输出 10 个原始分数供交叉熵计算；预测时取最大分数的类别，与先做 softmax 再取最大概率一致。



图 2: 按实际代码绘制的二维 CNN 结构图。青色表示特征提取，暖色表示分类部分。

CNN 共 206922 个可学习参数，与宽度 256 的 MLP 接近。因此二者的比较也有一定参考价值：参数数量相近时，利用图像空间结构的方式会影响识别效果。

4　结果与参数对比

4.1　标准测试集

表 1 列出六次实际训练的结果。时间包含各 epoch 的训练、验证与最佳权重保存，不含首次解压数据和额外测试。百分数均由程序记录的原始小数换算而来。

表 1: MNIST 实验结果；所有模型均按验证集最佳权重测试。

模型	参数量	最佳验证准确率	测试准确率	测试宏 F1	训练秒数
线性 baseline	7850	91.18%	92.22%	92.11%	9.03
MLP，宽度 128	101770	97.17%	97.59%	97.57%	9.08
MLP，宽度 256	203530	97.55%	97.70%	97.68%	8.83
MLP，宽度 256，低学习率	203530	95.97%	96.67%	96.64%	8.77
普通 CNN	206922	98.73%	98.78%	98.77%	12.20
增强 CNN	206922	98.60%	98.96%	98.96%	15.30

我首先把 MLP 宽度从 128 增至 256，学习率均为 0.001。验证准确率从 97.17% 升至 97.55%，所以后者被选为本实验的 MLP。保持宽度 256、把学习率改为 0.0003 后，在同样最多 6 轮的训练预算下验证准确率降至 95.97%。学习率变小使每步更新幅度变小，这组设置在 6 轮内还没有达到原设置的水平。说明可能学习率过大了导致优化不稳定。

图 3 显示每轮验证准确率和训练损失。普通 CNN 比 MLP 更早达到高准确率，线性模型则较早出现平台期。普通 CNN 和增强 CNN 在标准测试上的准确率相差 0.18 个百分点，增强 CNN 略高。

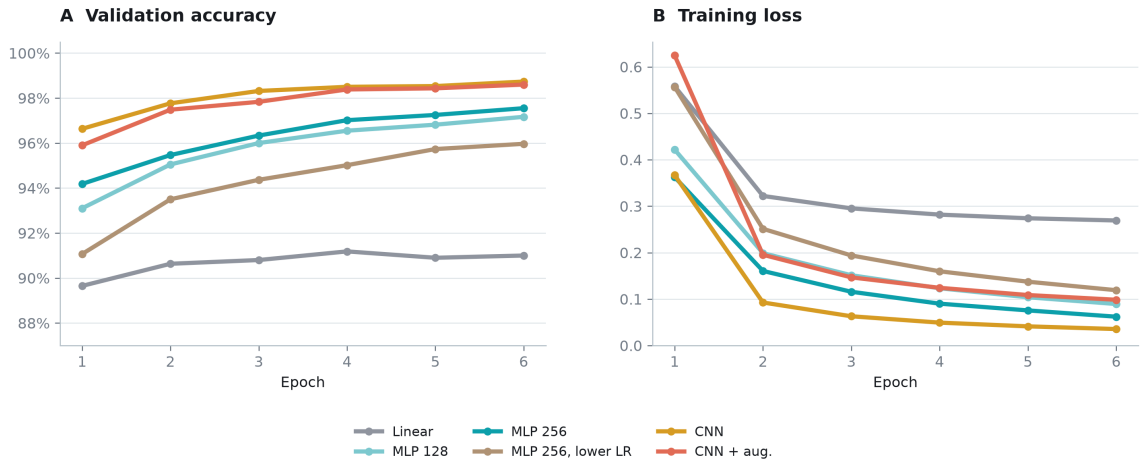


图 3: 六组实验的逐轮验证准确率和训练损失。

4.2　混淆情况

图 4 是增强 CNN 在标准测试集上的混淆矩阵。对角线越深，说明相应数字识别正确的数量越多。10000 张图片中有 9896 张被正确分类。较多的单向错分各有 6 张，例如真实数字 7 被预测成 9 或 2、真实数字 9 被预测成 4、真实数字 6 被预测成 1。这些都只是少量样本，仍说明形状相近的数字可能出现歧义。

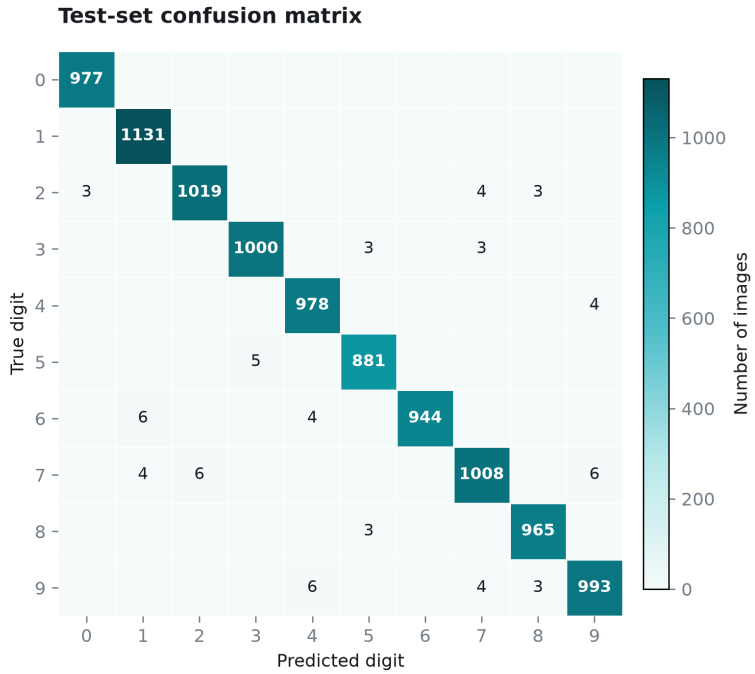


图 4: 增强 CNN 的 MNIST 测试集混淆矩阵，纵轴是真实类别，横轴是预测类别；非对角线仅标出不少于 3 张的单元格。

5　额外 baseline、数据增强和鲁棒性

线性分类器是加分实验中的额外 baseline。它的测试准确率达到 92.22%，说明很多数字仅凭像素位置也能区分；但与 MLP 的 97.70% 和 CNN 的 98.78% 相比，线性映射处理笔画变化的能力有限。

为了观察更困难的输入，我构造了两组补充测试。第一组对官方测试集每张图片固定旋转 15° 并平移 $(2, -2)$ 像素，共 10000 张。第二组用 Matplotlib 附带的三种 DejaVu 字体，每个数字在三个横向位置生成一张 28×28 灰度图，共 $3 \times 10 \times 3 = 90$ 张。它们是我自己生成的印刷体图片，和真实的手写照片不同；图 5 展示了三种字体在居中位置的全部数字。

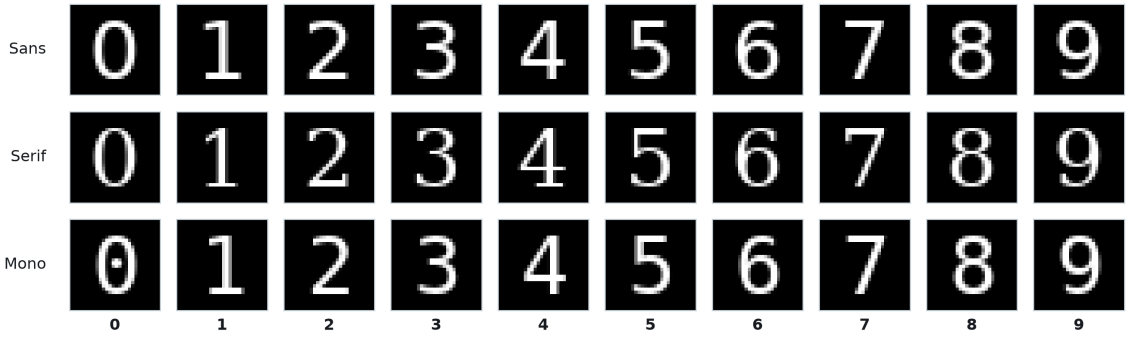


图 5: 自制字体数字示例，每行是一种字体，每列是一类数字。

表 2: 同一最佳权重在三组测试图片上的准确率。

模型	原始 MNIST（10000）	固定旋转平移（10000）	字体数字（90）
线性 baseline	92.22%	41.33%	51.11%
最佳 MLP	97.70%	66.28%	70.00%
普通 CNN	98.78%	82.76%	83.33%
增强 CNN	98.96%	95.63%	97.78%

增强 CNN 在固定旋转平移测试上比普通 CNN 高 12.87 个百分点，在字体数字上高 14.45 个百分点。训练时见过多种角度和位置的数字，可能使模型减少对某一个固定笔画位置的依赖。当然，字体测试只有 90 张，而且字体种类有限，所以这个结果只能说明模型对这组自制图片适应较好。如果想要更强的鲁棒性，可能还需要更多真实手写图片。

6 问题与思考

6.1 什么时候停止训练?

我设置最多 6 个 epoch，并保存验证准确率最高的一轮。若连续两轮没有刷新最高值，就提前停止。六组正式实验都在第 6 轮结束，其中线性分类器的最佳权重来自第 4 轮。由此可见，最后一轮的参数不一定就是最好的。

固定训练轮数容易安排时间，也方便不同实验在同一预算下比较。但不同模型收敛速度不同，固定轮数可能使某个模型训练不足，也可能使已经达到最佳验证效果的模型继续拟合训练集。验证集规则可以自动选权重并减少多余训练；它同时需要额外数据和每轮验证时间，若反复尝试很多参数，也可能逐渐对这一份验证集过度适应。因此我的参数搜索保持在较少的三组 MLP 设置。

6.2 参数怎样初始化?

本实验用 PyTorch 的默认层初始化：`Linear` 和 `Conv2d` 的默认权重使用与输入连接数相关的 Kaiming uniform 初始化，偏置使用相应范围的均匀分布。固定随机种子使同一种结构的初始随机状态可重复。初始值需要有一定随机性；若同一层所有权重都设成零，同层神经元接收相同梯度，难以学出不同特征。

零均值高斯初始化可让权重围绕 0 分布，但方差要结合层的输入、输出规模选择，过大可能使激活值变得很大，过小则可能让信号逐层衰减。ReLU 网络常用 He/Kaiming 初始化来照顾其只保留正半轴的特点；tanh 一类对称激活常用 Xavier 初始化。正交初始化让权重矩阵在若干方向上保持较独立的尺度，较深网络或循环网络中有价值。偏置通常可以初始化为零，因为权重已经打破了神经元之间的对称性。

6.3 怎样减少过拟合?

过拟合时，训练集继续变好，但验证集停滞甚至下降。我在 MLP 和 CNN 的全连接部分使用 0.2 的 Dropout，在增强 CNN 上对训练图片随机旋转和平移，还根据验证准确率挑选最佳权重。这三种方法分别减少对少数隐藏单元、固定图像位置和最后一轮参数的依赖。也可以尝试权重衰减、收集更多训练图片或缩小模型，但应查看验证集的变化再决定。本实验 CNN 第 6 轮训练准确率约 98.89%，验证准确率约 98.73%，没有出现很大的差距。

6.4 各模型有什么优缺点?

线性 baseline 只有 7850 个参数，结构容易理解，训练也快，但只能做一次线性映射。MLP 通过 ReLU 和隐藏层组合像素信息，本实验从线性模型的 92.22% 提升到 97.70%；展平图片后，它仍需从数据中学习图像邻近位置之间的关系。CNN 的局部卷积核和池化更适合数字笔画，参数量与最佳 MLP 接近时准确率达到 98.78%。它的计算过程稍复杂，训练时间也更长。增强 CNN 训练更慢、原始测试准确率比普通 CNN 高 0.18 个百分点，在本实验构造的位置变化和字体数字上表现更稳定。

6.5 换一批图片还可靠吗?

表 2 已经给出了两组补充测试。普通 CNN 在原始 MNIST 上是 98.78%，遇到固定旋转平移时降到 82.76%；这说明一个高标准测试分数不能单独说明模型能处理所有写法。增强 CNN 在该组上达到 95.63%，对这类已在训练中模拟过的变化适应更好。90 张字体图片给了另一个角度，但它们是印刷体，不是个人手写照片。下一步可以收集不同人用手机拍摄的真实数字，检查背景、笔画粗细和居中处理造成的影响。

7 心得与不足

这个实验让我把数据划分、模型选择和最终评估串起来理解了。开始时我只关注模型的准确率，后来发现验证集的用途同样重要：如果用测试集挑参数，最后的测试分数就难以代表未见数据的效果。线性模型、MLP 和 CNN 的逐步比较，也让我更直观地看到图像结构信息的作用。

目前只试了少量超参数，六轮预算也偏短；在字体图片上的结果不能代表真实手写照片。我保留了固定随机种子、全部实验记录和一条可运行命令，今后可以在相同划分上继续检查更多真实图片与其他增强方式。